

SETUP — the trinity

```
import * as THREE from 'three'

const scene = new THREE.Scene()
const camera = new THREE.PerspectiveCamera(
  75, innerWidth/innerHeight, 0.1, 1000)
camera.position.z = 5

const renderer = new THREE.WebGLRenderer({antialias:true})
renderer.setSize(innerWidth, innerHeight)
renderer.setPixelRatio(Math.min(devicePixelRatio,2))
document.body.appendChild(renderer.domElement)
```

Scene = world, Camera = view, Renderer -> canvas.

RENDER LOOP & RESIZE

```
const clock = new THREE.Clock()
function animate(){
  requestAnimationFrame(animate)
  const dt = clock.getDelta() // time-based!
  mesh.rotation.y += dt
  renderer.render(scene, camera)
}
animate()

addEventListener('resize', () => {
  camera.aspect = innerWidth/innerHeight
  camera.updateProjectionMatrix()
  renderer.setSize(innerWidth, innerHeight)
})
```

Multiply motion by delta. updateProjectionMatrix() after camera changes.

GEOMETRIES

```
new THREE.BoxGeometry(1,1,1)
new THREE.SphereGeometry(1, 32, 32)
new THREE.PlaneGeometry(10, 10)
new THREE.CylinderGeometry(1,1,2,32)
new THREE.TorusGeometry(1, 0.4, 16, 100)
new THREE.ConeGeometry(1, 2, 32)
// custom: BufferGeometry + setAttribute('position', ...)
```

MATERIALS

```
// no light, fastest:
new THREE.MeshBasicMaterial({color:0xff0000})
// PBR — the default:
new THREE.MeshStandardMaterial({
  color:0x4a7fd4, roughness:0.4, metalness:0.1})
// glass / clearcoat:
new THREE.MeshPhysicalMaterial({
  transmission:1, thickness:0.5})
```

StandardMaterial needs light, or it stays black.

MESH & TRANSFORMS

```
const mesh = new THREE.Mesh(geometry, material)
scene.add(mesh)

mesh.position.set(2, 0, -3)
mesh.rotation.y = Math.PI/4 // RADIANS, not degrees
mesh.scale.set(2, 2, 2)
mesh.lookAt(0, 0, 0)

const group = new THREE.Group() // scene graph
group.add(a, b); scene.add(group)
```

LIGHTS

```
scene.add(new THREE.AmbientLight(0xffffff, 0.4))
const sun = new THREE.DirectionalLight(0xffffff, 1)
sun.position.set(5, 10, 7)
scene.add(sun)
new THREE.PointLight(0xffaa00, 1, 20)
new THREE.SpotLight(0xffffff, 1)
new THREE.HemisphereLight(0x87ceeb, 0x442, 0.8)
```

Ambient + Directional covers 80%. Only Dir/Point/Spot cast shadows.

CAMERA & CONTROLS

```
import {OrbitControls} from
  'three/addons/controls/OrbitControls.js'

const controls = new OrbitControls(
  camera, renderer.domElement)
controls.enableDamping = true
// in loop: controls.update()

// PerspectiveCamera(fov, aspect, near, far)
// OrthographicCamera for isometric / 2D
```

TEXTURES

```
const t = new THREE.TextureLoader().load('/c.jpg')
t.colorSpace = THREE.SRGBColorSpace // color maps!
t.wrapS = t.wrapT = THREE.RepeatWrapping
t.repeat.set(8, 8)

new THREE.MeshStandardMaterial({
  map:t, normalMap:n, roughnessMap:r, aoMap:ao})
```

Color maps = SRGB. Data maps (normal/rough) = no color space.

LOAD MODELS — GLTF

```
import {GLTFLoader} from
  'three/addons/loaders/GLTFLoader.js'

new GLTFLoader().load('/model.glb', (gltf) => {
  scene.add(gltf.scene)
})
// .glb = packed glTF; use Draco/Meshopt to compress
```

glTF is the web standard. Get assets: Poly Haven, Sketchfab, Blender.

ANIMATION (mixer)

```
const mixer = new THREE.AnimationMixer(gltf.scene)
mixer.clipAction(gltf.animations[0]).play()
// in loop:
mixer.update(clock.getDelta())
// blend clips with action.crossFadeTo(next, 0.4)
```

SHADOWS

```
renderer.shadowMap.enabled = true
renderer.shadowMap.type = THREE.PCFSoftShadowMap
sun.castShadow = true
mesh.castShadow = true
floor.receiveShadow = true
sun.shadow.mapSize.set(2048, 2048)
sun.shadow.bias = -0.0001 // fix shadow acne
```

Enable in 3 places: renderer, light, objects.

ENVIRONMENT & FOG

```
scene.fog = new THREE.Fog(0xcccccc, 10, 50)
scene.background = new THREE.Color(0x87ceeb)

import {RGBELoader} from
  'three/addons/loaders/RGBELoader.js'
new RGBELoader().load('/sky.hdr', (hdr) => {
  hdr.mapping =
    THREE.EquirectangularReflectionMapping
  scene.environment = hdr // realistic lighting
})
```

An HDRI environment often replaces half your lighting setup.

RAYCASTING — click

```
const ray = new THREE.Raycaster()
const m = new THREE.Vector2()
addEventListener('click', (e) => {
  m.x = (e.clientX/innerWidth)*2 - 1
  m.y = -(e.clientY/innerHeight)*2 + 1
  ray.setFromCamera(m, camera)
  const hits = ray.intersectObjects(scene.children, true)
  if (hits[0]) hits[0].object.material.color.set('red')
})
```

PARTICLES (Points)

```
const pos = new Float32Array(count*3)
// fill pos with x,y,z values...
const g = new THREE.BufferGeometry()
g.setAttribute('position',
  new THREE.BufferAttribute(pos, 3))
const pts = new THREE.Points(g,
  new THREE.PointsMaterial({size:0.1}))
scene.add(pts)
```

PERFORMANCE

```
// many identical objects -> 1 draw call
const im = new THREE.InstancedMesh(geo, mat, 1000)
im.setMatrixAt(i, matrix) // per instance

renderer.setPixelRatio(Math.min(devicePixelRatio,2))
geometry.dispose(); material.dispose() // cleanup!
// THREE.LOD for distance-based detail
// shrink textures; KTX2/Basis for big projects
```

Measure with stats.js first. Usual bottleneck: draw calls or texture size.

COLOR & TONE MAPPING

```
renderer.toneMapping =
  THREE.ACESFilmicToneMapping
renderer.toneMappingExposure = 1.0
// modern three.js outputs sRGB by default
```

ACES tone mapping = a softer, photographic look.

WEBGPU (2026)

```
import * as THREE from 'three/webgpu'
const renderer = new THREE.WebGPURenderer({antialias:true})
await renderer.init()
// auto-fallback to WebGL when unsupported
// custom shaders -> port to TSL / node materials
```

Production-ready since r171 + Safari 26. Adds compute shaders.

REACT THREE FIBER

```
import {Canvas, useFrame} from '@react-three/fiber'
import {OrbitControls} from '@react-three/drei'
```

```
<Canvas camera={{position:[0,0,5]}}>
  <ambientLight intensity={0.6} />
  <directionalLight position={[5,5,5]} />
  <mesh>
    <boxGeometry args={[1,1,1]} />
    <meshStandardMaterial color="#4a7fd4" />
  </mesh>
  <OrbitControls enableDamping />
</Canvas>
```

Drei = helpers, Rapier = physics. The React way to do Three.js.

HELPERS & DEBUG

```
scene.add(new THREE.AxesHelper(5))
scene.add(new THREE.GridHelper(20, 20))
new THREE.DirectionalLightHelper(sun)
new THREE.CameraHelper(camera)
```

```
import GUI from 'lil-gui' // live tweak panel
new GUI().add(sun, 'intensity', 0, 5)
```

lil-gui + stats.js = your two best debugging friends.